01 Raj

Introduction, motivation and research justification

Fuzzy-Logic-Based QoS Control in oneM2M (Zyane & Et-Tousy, FiCloud 2026)

Your job

You open the talk. In about **4 minutes** the room should know: what problem the paper attacks (busy IoT servers), why the usual fix (fixed thresholds) is not good enough, what gap the paper fills, and the three headline numbers. You do *not* explain fuzzy logic or the architecture; Manoj and Ashutosh do.

Your slides

Slides 1–7: title, roadmap, bursty traffic and what QoS means, kitchen analogy, problem statement, related work and the gap, contributions.

Opening line

“Good morning. We are presenting a 2026 FiCloud paper on a simple question: when an IoT server gets too busy, how much of its work should it hand over to the cloud?”

Hand-off line

“So that is the why. Manoj will now show you the system the authors built.”

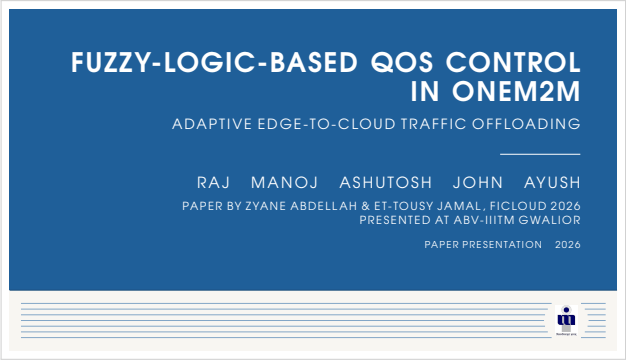
Numbers you must know

43.5 %	lower total cost (\$2.17 $\rightarrow$ \$1.22)
55.9 %	lower round-trip time (8.5 s $\rightarrow$ 3.75 s)
19.4 %	higher success rate (77.5 % $\rightarrow$ 92.5 %)
27	fuzzy rules (3 inputs $\times$ 3 levels each $= 3^3$)
5 s	how often the controller re-decides

Words to say correctly

oneM2M = “one-M-two-M” (machine-to-machine). **QoS** = Quality of Service. **MAPE-K** = “mape-kay”: Monitor, Analyze, Plan, Execute + Knowledge. **Offloading** = sending work to the cloud.

Slide 1 ~20 s

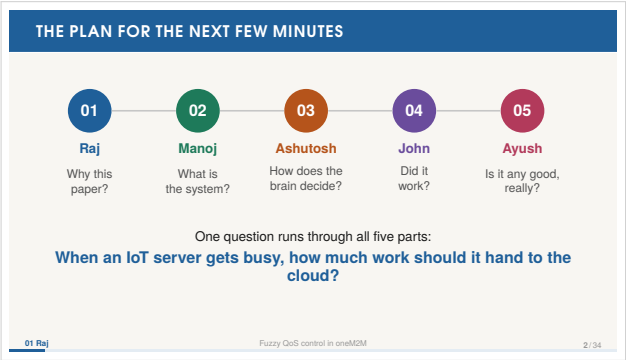


Title slide. Paper title, the five of us, original authors and venue.

Say: "Good morning. We are presenting *Fuzzy-Logic-Based QoS Control in oneM2M: Adaptive Edge-to-Cloud Traffic Offloading*, by Zyane Abdellah and Et-Tousy Jamal from Cadi Ayyad University in Morocco, published at FiCloud 2026, the IEEE conference on Future Internet of Things and Cloud."

Say: "I'm Raj, and with me are Manoj, Ashutosh, John and Ayush."

Slide 2 ~25 s



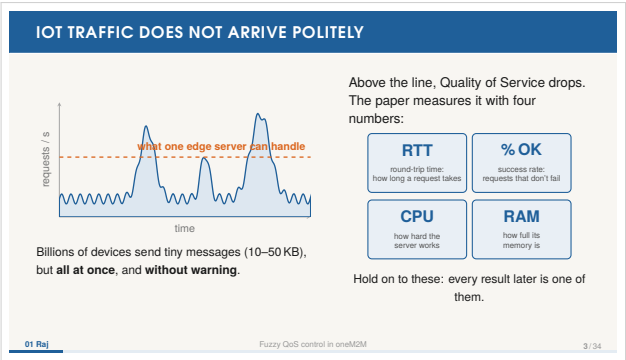
Roadmap: five coloured circles, one per presenter, and the one question that ties the talk together.

Say: "We split the paper into five questions. I'll tell you why the paper exists. Manoj shows the system. Ashutosh opens the brain of it, the fuzzy controller. John shows whether it worked, and Ayush tells you whether it is actually any good."

Say: "Keep one question in mind for the whole talk: *when an IoT server gets busy, how much work should it hand to the cloud?*"

Tip: Each part has its own colour. Point out that the colour bar at the top changes with the speaker.

Slide 3 ~50 s



Left: a bursty traffic curve crossing a dashed "capacity" line. Right: the four QoS numbers, RTT, success rate, CPU, RAM.

Say: "IoT is everywhere: factories, hospitals, smart cities, wearables. Each device sends tiny messages, 10 to 50 kilobytes. The problem is not size. It is *timing*. When something happens, many devices talk at

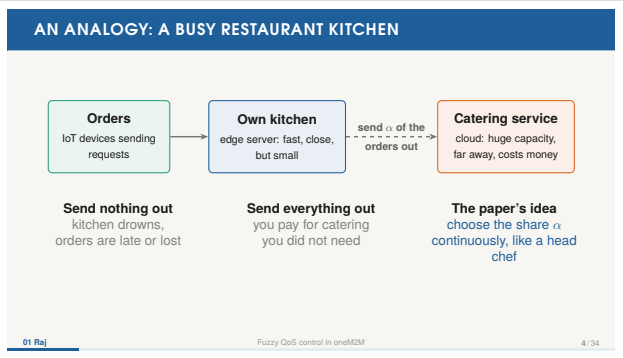
once.”

Say: “The dashed line is what one edge server can handle. Every time the traffic goes above it, requests queue up, delays explode, and some requests fail.”

Say: “Above that line, Quality of Service drops. QoS sounds abstract, so here, on the right, is what the paper actually measures. Round-trip time: how long a request takes. Success rate: how many requests do not fail. And CPU and RAM: how hard the server is working.”

Say: “Every result you will see later is one of these four numbers.”

Slide 4 ∼35 s



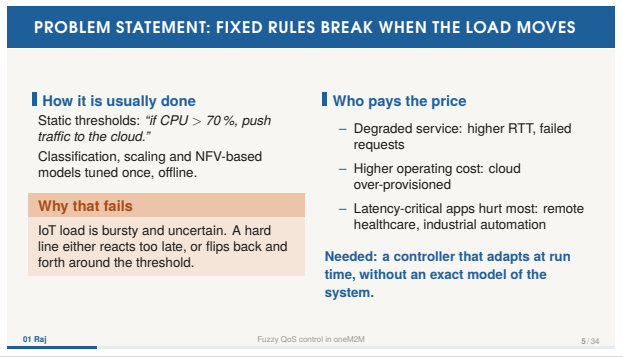
Restaurant analogy: orders → own kitchen → catering service, with α as the share sent out.

Say: “Think of a restaurant. The kitchen is the edge server: close and fast, but small. A catering company is the cloud: huge, but far away, and you pay for it.”

Say: “If you send nothing out on a busy night, the kitchen drowns. If you send everything out, you pay for catering you did not need. The smart move is to decide, minute by minute, *what share* to send out. The paper calls that share **alpha**, and the whole paper is about choosing alpha well.”

Tip: This analogy is reused by John later. Say “alpha” clearly.

Slide 5 ∼35 s



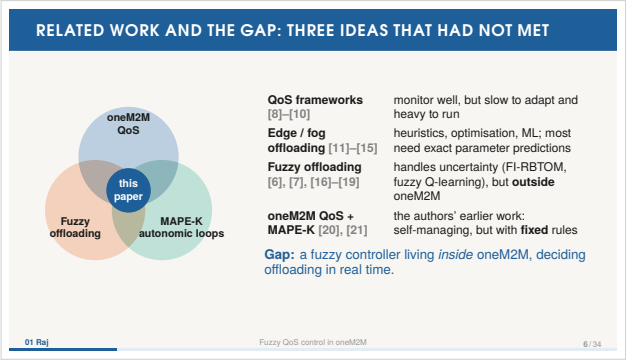
Problem statement: static thresholds, why they fail, who pays.

Say: “Today this is usually done with fixed thresholds: if CPU goes above 70 %, push traffic to the cloud. Or with models that are tuned once, offline.”

Say: “IoT load does not respect fixed lines. A hard threshold either reacts too late, or it flips back and forth around the line. The price is slower service, failed requests, and a bigger cloud bill, and it hurts most in latency-critical uses like remote healthcare and industrial automation.”

Say: “So we need a controller that adapts while the system runs, without needing an exact mathematical model of it.”

Slide 6 ~55 s



Venn diagram of oneM2M QoS, fuzzy offloading and MAPE-K loops, with the paper in the middle; on the right, the four strands of related work.

Say: "Three lines of work already exist. People have studied QoS in oneM2M, but with fixed rules. People have used fuzzy logic for offloading, but outside oneM2M. And people have built self-managing MAPE-K loops for oneM2M, but without fuzzy reasoning."

Say: "Nobody had put all three together: a fuzzy controller living *inside* the oneM2M middleware, deciding offloading in real time. That intersection is this paper."

Say: "On the right, the related work in a bit more detail. QoS frameworks for IoT monitor well but adapt slowly. Edge and fog offloading methods mostly need exact predictions of parameters, which IoT rarely gives you. Fuzzy offloading, like FI-RBTOM and fuzzy Q-learning, handles uncertainty well. And the authors' own earlier papers built SLA-driven traffic control in oneM2M."

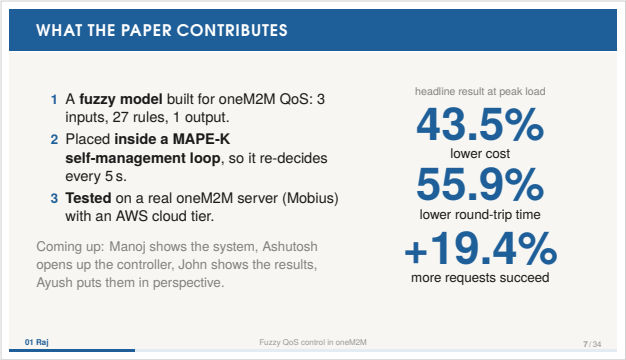
Say: "Each line solves part of the problem; none does all of it inside oneM2M."

Tip: Do not read the reference numbers aloud. They are there for anyone checking.

Q: *Is this really new?*

A: The individual pieces are not new; the paper's claim is the integration inside a standards-compliant oneM2M server. Ayush comes back to how strong that claim is.

Slide 7 ~35 s



Three contributions on the left, three headline numbers on the right.

Say: "The paper contributes three things: a fuzzy model built for oneM2M QoS, with three inputs and 27 rules; that model placed inside a self-management loop that re-decides every five seconds; and a test on a real oneM2M server, Mobius, with an Amazon cloud tier."

Say: "And the headline: at peak load, compared with doing nothing, cost goes down 43.5 %, round-trip time goes down 55.9 %, and 19.4 % more requests succeed."

Say: "So that is the why. Manoj will now show you the system the authors built."

If you get questions during Q&A

Q: *Why not just buy a bigger edge server?*

A: Load is bursty: a server big enough for the peak sits idle most of the time. Offloading only the overflow is cheaper.

Q: *Why not machine learning, or just a neural net?*

A: There is no labelled dataset of "the right offloading share" for a real server, so you would train on made-up data, and you would have to retrain every time the setup changes. Fuzzy rules work from day one and an operator can tweak them directly. Ashutosh has a whole slide on this (slide 13).

Q: *What is oneM2M exactly?*

A: A global standard for IoT middleware, a common service layer between devices and apps. Manoj's first slide covers it.